

Perfect 😊 Let's continue.

🚀 Lesson 6.7 — RSC Payload, Flight & Reconciliation

Now we're going to answer the question we parked:

If Server Components don't send their JavaScript to the browser, how does React know what the Server Component tree looks like?

The answer is **RSC Payload**, often called the **React Flight payload/protocol**.

1. The core problem

Imagine:

```
export default async function ProductPage() {  
  const product = await getProduct();  
  
  return (  
    <>  
    <h1>{product.name}</h1>  
  )  
}
```

```
<AddToCartButton id={product.id} />
</>
);
}
```

We have:

```
ProductPage
  ↓
Server Component

AddToCartButton
  ↓
Client Component
```



The browser doesn't need:

```
✗ ProductPage.js
✗ getProduct.js
✗ database code
```



Those execute on the server.

But the browser/React still needs to know:

```
"There's an AddToCartButton here"
"Here are its props"
"Here's where it belongs in the tree"
```



That's what the RSC payload helps communicate.

2. Think of RSC payload as instructions/data for React

Very simplified:

```
Server Component Tree
  ↓
React Server Rendering
  ↓
RSC / Flight payload
  ↓
Browser React
```



Conceptually, the payload says something like:

```
Main
├── h1
│   └── "iPhone"
└── Client Component
    └── AddToCartButton
        props:
            id = "123"
```



⚠ That's **not the literal wire format**. It's just a mental model.

3. Why is this useful?

Because React can use that information to construct/reconcile the component tree **without shipping the Server Component's source code to the browser**.

That's the beautiful part.

Server:



```
ProductPage()
  ↓
database
  ↓
React Server Component tree
  ↓
RSC payload
```

Browser:

```
RSC payload
  ↓
React understands tree/result
  ↓
Client Components remain interactive
```



4. Now navigation makes much more sense

Suppose you're on:

```
/products/123
```



and click:

```
<Link href="/products/456">  
  Product 456  
</Link>
```

The browser doesn't necessarily need a whole new document.

Instead:

```
Click Link  
  ↓  
Next.js Client Router  
  ↓  
request /products/456  
  ↓  
Server renders relevant Server Components  
  ↓  
RSC payload  
  ↓  
Browser  
  ↓  
React reconciles  
  ↓  
UI changes
```



So:

```
/products/123  
  ↓  
  RSC  
  ↓  
React tree  
  ↓  
/products/456
```



5. Where Router Cache fits

You already learned this.

Suppose:

```
/products/123
```



was visited.

The client router can retain the RSC result in the **Router Cache**.

Then:

Router Cache

↓

RSC payload / route segments



So when navigating around the app, Next.js can reuse client-side information where applicable rather than always starting from zero.

6. What happens when data changes?

Suppose:

Product 123
price = ₹50,000



Then someone changes it:

price = ₹55,000



Server-side cache might need invalidation/revalidation.

You've already learned:

```
cacheTag()  
  ↓  
revalidateTag()  
  ↓  
cached result needs revalidation
```



or:

```
updateTag()  
  ↓  
invalidate immediately
```



Then a subsequent render/navigation can obtain the updated Server Component result.

Conceptually:

Old cached result
₹50,000



```
↓  
invalidate  
↓  
rerun server computation  
↓  
₹55,000  
↓  
new RSC result  
↓  
browser
```

7. This also explains `router.refresh()`

You asked about this earlier.

Suppose you're currently viewing:

`/products/123`



and execute:

```
router.refresh();
```

The browser tells Next.js:

"Give me a fresh Server Component result for this route."

Conceptually:

```
router.refresh()  
↓  
request current route's RSC result  
↓  
Server  
↓  
Server Components execute/reuse valid cache  
↓  
new RSC payload  
↓  
Browser  
↓  
React reconciles
```



Notice:

```
router.refresh()  
≠  
window.location.reload()
```



It's specifically tied into the App Router/RSC model.

8. Reconciliation is the next piece

React receives the new Server Component result and compares it with what it already has.

Conceptually:

OLD TREE

```
Product  
  price ₹50k  
  Cart 2
```

↓ new RSC result

NEW TREE

```
Product  
  price ₹55k  
  Cart 2
```



React determines what needs to change.

It doesn't blindly throw away the entire browser application and start over.

That's why client state can often be preserved across App Router updates.

9. Server Components vs Client Components

This distinction is now much clearer:

Server Component

```
Server  
↓  
execute  
↓
```



RSC result



browser

Its implementation doesn't need to be sent to the browser for execution.

Client Component

Server



RSC identifies Client Component



browser gets component JS



React loads it



hydration/interactivity



So:

Server Component

→ server execution

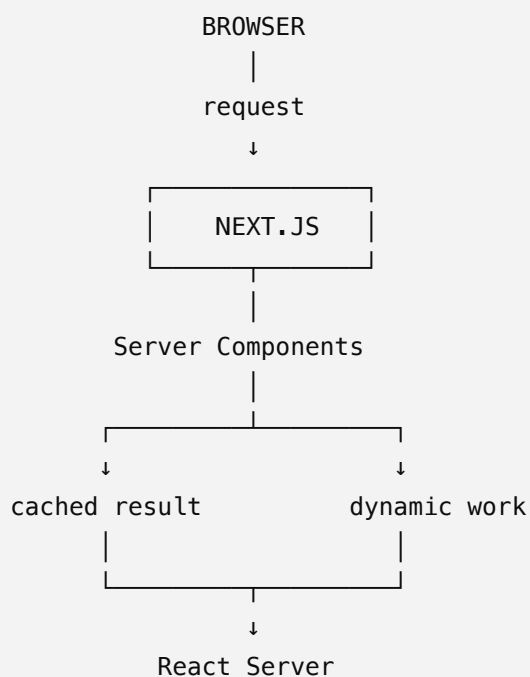
Client Component

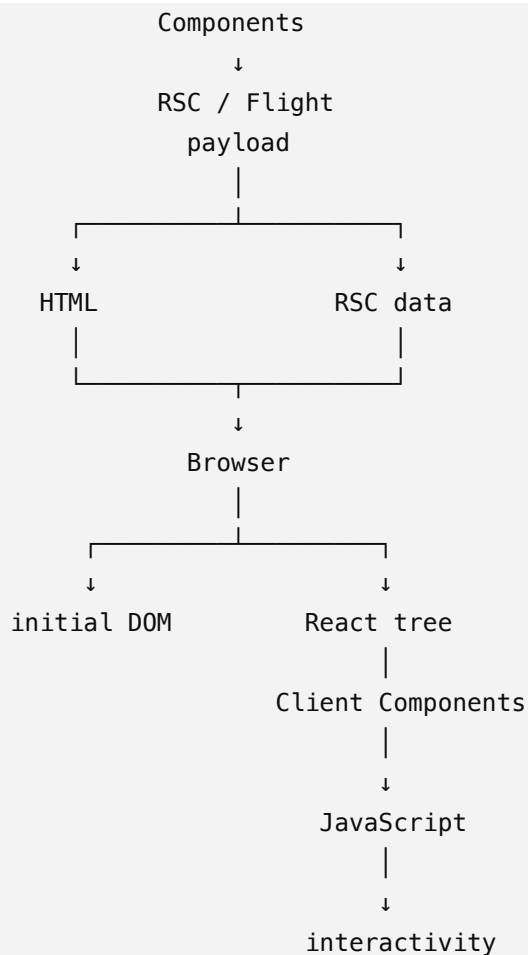
→ browser execution



10. Complete flow 🔥

Here's your architecture now:





Interview version

If someone asks:

"What is the RSC payload?"

You can say:

"The RSC payload is the serialized representation of the React Server Component result/tree produced on the server. It allows React and Next.js to transfer Server Component results to the client without shipping the Server Component implementation to the browser. It's also important for App Router navigation and updating the existing React tree."

That's a strong answer.

 **And now your whole Next.js mental model is becoming one system**

Routing

↓



```
Rendering
  ↓
Server Components
  ↓
"use cache"
  ↓
PPR
  ↓
RSC payload
  ↓
HTML + RSC
  ↓
Browser
  ↓
Router Cache
  ↓
Client Components + JS
  ↓
Hydration / interactivity
```

The **next logical topic** is **hydration vs reconciliation vs RSC update** — specifically, *what happens to your Client Component state when `router.refresh()` or navigation occurs*. That's a very common interview trap.



wait if servers component code is not there in browser
how does it still show on screen ??also can i see in
browser where is router cache like visually and is it
possible to see cached server component visually from
laptop ??...

Yes — this is the **exact confusion to clear up**. 🤪 Once you understand this, Server Components become much less magical.

1. "If Server Component code isn't in the browser, how does it show on screen?"

Because the browser doesn't need the Server Component code to display its result.

Think about a restaurant 🍕 :

```
Chef (server)
  ↓
uses recipe + ingredients
```



↓
prepares pizza
↓
waiter delivers pizza
↓
Customer sees pizza

The customer doesn't need the **recipe** or the **kitchen** to see the pizza.

Same idea:

SERVER
Server Component code
↓
executes
↓
produces result
↓
HTML / RSC payload
↓
BROWSER
↓
screen



2. Concrete example

You write:

```
</> TypeScript

// app/products/page.tsx

export default async function ProductsPage() {
  const products = await db.product.findMany();

  return (
    <main>
      <h1>Products</h1>

      {products.map(product => (
        <div key={product.id}>
          {product.name}
        </div>
      ))}
    </main>
  );
}
```



This is a **Server Component**.

The browser does NOT need:

- ✗ ProductsPage.tsx
- ✗ db.product.findMany()
- ✗ database credentials
- ✗ database connection



The server executes:

```
ProductsPage()  
  ↓  
DB query  
  ↓  
React renders result
```



Suppose the result is:

```
Products:  
iPhone  
MacBook  
AirPods
```



The browser can receive/render markup representing that result:

```
HTML  
  
<main>  
  <h1>Products</h1>  
  <div>iPhone</div>  
  <div>MacBook</div>  
  <div>AirPods</div>  
</main>
```



So the browser says:

"Cool, I have markup. I'll paint this on the screen."

It doesn't need to know **how the server produced it**.

3. Then what is RSC doing?

Here's the important distinction:

Server Component CODE



runs on server



Server Component RESULT



RSC payload



browser React



The browser receives the **result/representation**, not the original server implementation.

That's why:

✗ Browser:

"Let me execute ProductsPage.tsx"

vs

✓ Browser:

"I received the result of ProductsPage."



4. Can you SEE Router Cache in your browser? 🙄

Yes — but there's an important caveat.

You won't normally open DevTools and see:

Router Cache

/products/123

/products/456



as a nice readable cache panel.

Next.js doesn't expose Router Cache as a simple Chrome DevTools storage bucket like:

Application

├ Local Storage

├ Session Storage

├ IndexedDB

└ Router Cache ← ✗ not normally here



Router Cache is an **internal client-side cache** maintained by the Next.js App Router, not ordinary `localStorage`.

5. But you CAN inspect evidence of it 🔍

This is actually a great experiment for you.

Open your Next.js app in Chrome.

Then:

Chrome DevTools



Network



Navigate using:

</> TypeScript

```
<Link href="/products/123">
```



rather than:

HTML

```
<a href="/products/123">
```



You can inspect the network requests and see requests associated with the App Router/RSC navigation.

Depending on your Next.js version and dev/prod mode, you'll see RSC-related request/response information.

You may encounter headers such as:

RSC

Next-Router-State-Tree

Next-Router-Prefetch



and responses associated with the RSC protocol.

The exact presentation varies between development/production and Next.js versions.

6. Here's a really good experiment

Make two pages:

```
/products/123  
/products/456
```



and navigate:

```
123  
↓  
456  
↓  
123
```



using:

```
</> TypeScript  
  
<Link href="/products/123">
```



Watch:

```
DevTools  
→ Network
```



You'll be able to observe the App Router making RSC/navigation requests.

Then try:

```
/products/123
```



→

```
/products/456
```



→

```
/products/123
```



The Router Cache can allow previously received route information to be reused according to its current caching behavior.

7. Can you visually see the cached Server Component?

This needs a careful distinction.

✗ You generally won't see:

Next.js Server Cache



Product component
cached result

inside Chrome DevTools.

Why?

Because that cache is **on the server**, not in your browser.

Remember:

"use cache"



↓
SERVER-SIDE CACHE

while:

Router Cache



↓
CLIENT-SIDE

They're different.

8. So where can you inspect each one?

Thing	Where it lives	Can you inspect it?
Server Component source	Server/build	Not as browser JS
"use cache" result	Server-side cache	Not directly through normal browser DevTools
RSC response	Network	✔ Yes
HTML	Network → document response	✔ Yes
Client JS	Network/Sources	✔ Yes
Router Cache	Next.js client runtime	⚠ Not as a normal DevTools storage panel

Thing	Where it lives	Can you inspect it?
localStorage	Browser	✅ Yes
sessionStorage	Browser	✅ Yes
HTTP/browser cache	Browser DevTools	✅ Partially inspectable

9. And here's a SUPER useful experiment for you

Open:

Chrome DevTools
→ Network
→ click the document request
→ Response



You'll be able to inspect the initial HTML response.

Then navigate with `<Link>` and inspect the subsequent requests.

You'll start seeing the difference between:

INITIAL LOAD

HTML
+
RSC information
+
Client JS



and:

CLIENT NAVIGATION

RSC-related response
+
existing browser document



That distinction is **really important for App Router interviews**.

10. One more thing: don't confuse "showing" with "executing"

This is the key sentence:

A browser can display the output of code without possessing or executing the code that generated that output.

For example:

Server Component:

```
<h1>{product.name}</h1>
  ↓
server knows product.name = "iPhone"
  ↓
result
  ↓
<h1>iPhone</h1>
  ↓
browser displays it
```

The browser doesn't need:

TS TypeScript

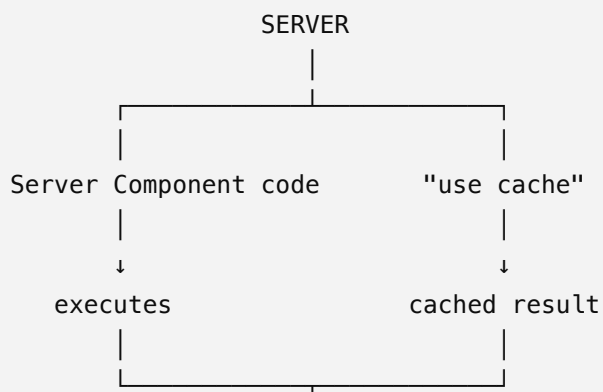
```
<h1>{product.name}</h1>
```

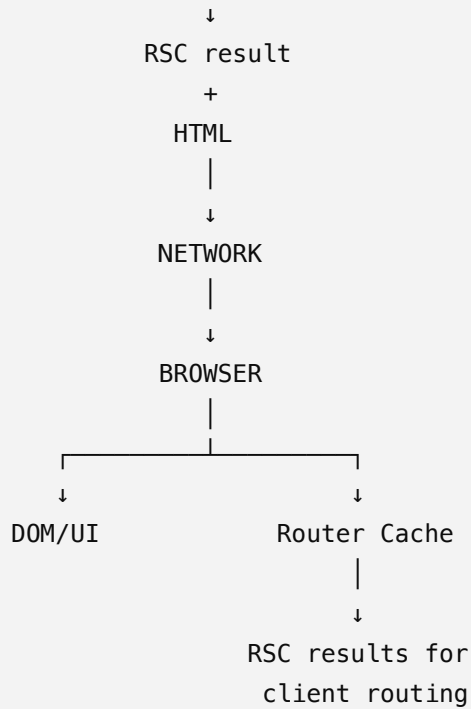
to be able to display:

iPhone

Final picture

Keep these two worlds separate:





🔥 The one thing to remember:

Server Component code stays on the server. Its *result* crosses the network.

And yes, **you can absolutely open DevTools and inspect the HTML and RSC network traffic.** You just generally can't open a little "Router Cache" folder and browse it like localStorage.

If you want, we can next do a **hands-on DevTools experiment** where I give you a tiny Next.js page and tell you **exactly what to click in Chrome DevTools and what you should expect to see.** That'll make this stick for real. 🙄



i willdo it after i build nextjs app ..next

Absolutely 🤪 We'll do the DevTools experiment **after you build the Next.js app** so you can actually see the HTML, RSC requests, navigation, and caching instead of just imagining them.

Next → Lesson 6.8 🚀

We'll move into:

Hydration vs Reconciliation vs RSC Updates

We'll answer:

Server Component renders on server

↓

HTML reaches browser

↓

Client Component JS arrives

↓

What exactly gets hydrated?

↓

What does React reconcile?

↓

What happens during router.refresh()?

↓

What happens during <Link> navigation?



And we'll use one small example throughout so the differences are crystal clear. 🧠



how many more?

Haha 😄 you're asking **how many lessons are left in the whole Next.js roadmap**, right?

You completed **Lesson 4**, and we're currently around **Lesson 6.7** in the deeper caching/RSC section.

Remaining roadmap

Lesson	Topic	Status
6.8	Hydration vs Reconciliation vs RSC Updates	➡️ SOON Next
6.9	Full request lifecycle — putting everything together	🕒
7	Next.js Performance	🕒
8	Authentication & Authorization	🕒
9	Middleware / Proxy & request interception	🕒
10	Route Handlers & APIs	🕒

Lesson	Topic	Status
11	Error Handling & Loading UX	🕒
12	Forms, Server Actions & Mutations — interview deep dive	🕒
13	SEO, Metadata & OG Images	🕒
14	Images, Fonts & Scripts	🕒
15	Deployment, Vercel, CDN & Edge	🕒
16	Testing Next.js apps	🕒
17	Security & production best practices	🕒
18	Next.js System Design	🕒
19	Interview questions + debugging scenarios	🕒
20	Build the Web Performance Analyzer 🚀	🕒

So roughly 14 major lessons/sections remain.

But don't worry — several of those are much smaller than the caching/RSC section we've just gone through. 😂

And 6.8 → 6.9 is basically the final deep architectural piece. After that we'll move into practical/interview-focused Next.js topics.

Your journey so far

Lesson 1 → Server vs Client Components

Lesson 2 → App Router & Routing

Lesson 3 → Navigation & Routing details

Lesson 4 → Data Fetching

Lesson 5 → Mutations & Server Actions

Lesson 6 → Caching + Rendering architecture

- └ Cache Components
- └ use cache
- └ cacheLife
- └ cacheTag
- └ PPR
- └ RSC
- └ 6.8 Hydration/Reconciliation

✓

✓

✓

✓

✓

🔄

✓

✓

✓

✓

✓

✓

➡
SOON

📄

So we're **well past the fundamentals now.** 🔥

And after the roadmap, we'll use all of this in your **Web Performance Analyzer project**, which is where the concepts should really become second nature.



ChatGPT can make mistakes. Check important info.